DeepL
Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.



Chương 3 – Phát triển phần mềm theo phương pháp Agile

Tác giả: Ian Sommerville ©2014
Người trình bày: Vũ Thị Trà

1

Mục tiêu



Sau khi hoàn thành chương này, bạn sẽ:

- hiểu được cơ sở lý luận của các phương pháp phát triển phần mềm linh hoạt, Tuyên ngôn Agile và sự khác biệt giữa phát triển linh hoạt và phát triển theo kế hoạch;
- biết về các thực hành phát triển linh hoạt quan trọng như câu chuyện người dùng, tái cấu trúc mã, lập trình cặp và phát triển theo phương pháp kiểm thử trước;
- hiểu phương pháp Scrum trong quản lý dự án linh hoạt;
- hiểu các vấn đề liên quan đến việc mở rộng quy mô các phương pháp phát triển linh hoạt và kết hợp các phương pháp linh hoạt với các phương pháp dựa trên kế hoạch trong việc phát triển các hệ thống phần mềm quy mô lớn.

2

Các chủ đề được đề cập



- Các phương pháp Agile
- Các kỹ thuật phát triển Agile
- Quản lý dự án Agile
- Mở rộng quy mô các phương pháp Agile

3

Phát triển phần mềm nhanh



- Phát triển và triển khai nhanh chóng hiện nay thường là yêu cầu quan trọng nhất đối với các hệ thống phần mềm
  - Các doanh nghiệp hoạt động trong môi trường yêu cầu thay đổi nhanh chóng và việc đưa ra một bộ yêu cầu phần mềm ổn định là điều gần như không thể
  - Phần mềm phải phát triển nhanh chóng để phản ánh những nhu cầu kinh doanh đang thay đổi.
- Phát triển theo kế hoạch là điều cần thiết đối với một số loại hệ thống, nhưng không đáp ứng được những nhu cầu kinh doanh này.
- Các phương pháp phát triển linh hoạt (Agile) xuất hiện vào cuối những năm 1990 với mục tiêu giảm đáng kể thời gian triển khai các hệ thống phần mềm hoạt động

4

Phát triển linh hoạt

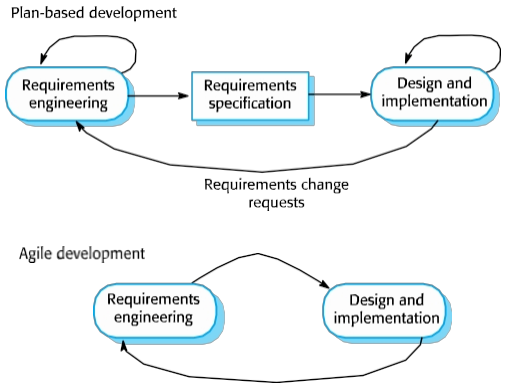


- Việc xác định yêu cầu, thiết kế và triển khai chương trình được thực hiện xen kẽ
- Hệ thống được phát triển dưới dạng một chuỗi các phiên bản hoặc các đợt tăng cường, với sự tham gia của các bên liên quan trong việc xác định yêu cầu và đánh giá phiên bản
- Thường xuyên phát hành các phiên bản mới để đánh giá
- Sử dụng hỗ trợ công cụ rộng rãi (ví dụ: công cụ kiểm thử tự động) để hỗ trợ quá trình phát triển.
- Tài liệu tối thiểu – tập trung vào mã nguồn hoạt động

5

Phát triển theo kế hoạch và phát triển linh hoạt





6

Phát triển theo kế hoạch và phát triển linh ho



- Phát triển theo kế hoạch
  - Cách tiếp cận phát triển phần mềm theo kế hoạch dựa trên các giai đoạn phát triển riêng biệt, trong đó các sản phẩm đầu ra cần tạo ra ở mỗi giai đoạn đều được lên kế hoạch trước.
  - Không nhất thiết phải là mô hình thác nước – phát triển theo kế hoạch và từng bước vẫn có thể thực hiện được
  - Các lần lặp lại diễn ra trong các hoạt động.
- Phát triển linh hoạt
  - Việc xác định yêu cầu, thiết kế, triển khai và kiểm thử được xen kẽ với nhau, và các sản phẩm đầu ra từ quá trình phát triển được quyết định thông qua quá trình đàm phán trong suốt quá trình phát triển phần mềm.

7



Các phương pháp Agile

8

Các phương pháp Agile



- Sự không hài lòng với các chi phí phát sinh liên quan đến các phương pháp thiết kế phần mềm của những năm 1980 và 1990 đã dẫn đến sự ra đời của các phương pháp Agile. Các phương pháp này:
  - Tập trung vào mã nguồn thay vì thiết kế
  - Dựa trên phương pháp tiếp cận lặp trong phát triển phần mềm
  - Được thiết kế để cung cấp phần mềm hoạt động nhanh chóng và nhanh chóng điều chỉnh để đáp ứng các yêu cầu thay đổi.
- Mục tiêu của các phương pháp Agile là giảm thiểu gánh nặng trong quy trình phát triển phần mềm (ví dụ: bằng cách hạn chế tài liệu) và có khả năng phản ứng nhanh chóng với các yêu cầu thay đổi mà không cần phải làm lại quá nhiều.

Tuyên ngôn Agile



- *Chúng tôi đang khám phá những cách tốt hơn để phát triển phần mềm thông qua việc thực hành và hỗ trợ người khác thực hiện. Qua quá trình này, chúng tôi đã nhận ra giá trị của:*
  - *Cá nhân và sự tương tác hơn là quy trình và công cụ; Phần mềm hoạt động hơn là tài liệu toàn diện; Hợp tác với khách hàng hơn là đàm phán hợp đồng; Phản ứng với sự thay đổi hơn là tuân theo kế hoạch*
- *Tức là, mặc dù các yếu tố ở bên phải cũng có giá trị, nhưng chúng tôi coi trọng các yếu tố ở bên trái hơn.*

Các nguyên tắc của phương pháp Agile



Nguyên tắc	Mô tả
Sự tham gia của khách hàng	Khách hàng nên được tham gia chặt chẽ trong suốt quá trình phát triển. Vai trò của họ là đưa ra và ưu tiên các yêu cầu mới của hệ thống, đồng thời đánh giá các phiên bản lặp của hệ thống.
Phát hành theo từng giai đoạn	Phần mềm được phát triển theo từng giai đoạn, trong đó khách hàng xác định các yêu cầu cần được đưa vào mỗi giai đoạn.
Con người chứ không phải quy trình	Kỹ năng của đội ngũ phát triển cần được công nhận và phát huy. Các thành viên trong đội nên được tự do phát triển cách làm việc riêng của mình mà không bị ràng buộc bởi các quy trình cứng nhắc.
Chấp nhận sự thay đổi	Dự kiến các yêu cầu hệ thống sẽ thay đổi và do đó thiết kế hệ thống để thích ứng với những thay đổi này.
Giữ sự đơn giản	Tập trung vào sự đơn giản cả trong phần mềm đang được phát triển lẫn trong quy trình phát triển. Bất cứ khi nào có thể, hãy chủ động loại bỏ sự phức tạp khỏi hệ thống.

Khả năng áp dụng phương pháp Agile



- Phát triển sản phẩm trong trường hợp một công ty phần mềm đang phát triển một sản phẩm quy mô nhỏ hoặc vừa để bán.
  - Hiện nay, hầu như tất cả các sản phẩm phần mềm và ứng dụng đều được phát triển theo phương pháp Agile
- Phát triển hệ thống tùy chỉnh trong nội bộ tổ chức, nơi khách hàng cam kết rõ ràng sẽ tham gia vào quá trình phát triển và nơi có ít quy định và quy tắc bên ngoài ảnh hưởng đến phần mềm.



Các kỹ thuật phát triển linh hoạt

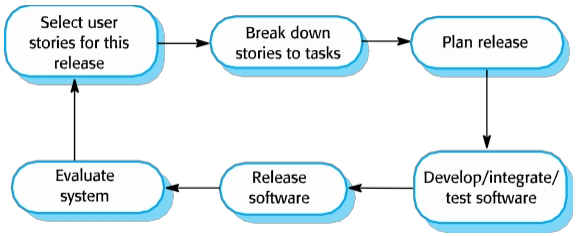


Lập trình cực đoan

- Một phương pháp Agile rất có ảnh hưởng, được phát triển vào cuối những năm 1990, đã giới thiệu một loạt các kỹ thuật phát triển Agile.
- Lập trình Cực đoan (XP) áp dụng một cách tiếp cận "cực đoan" đối với phát triển lặp.
  - Các phiên bản mới có thể được xây dựng nhiều lần trong ngày;
  - Các bản tăng dần được giao cho khách hàng mỗi 2 tuần;
  - Tất cả các bài kiểm thử phải được chạy cho mỗi lần xây dựng và bản xây dựng chỉ được chấp nhận nếu các bài kiểm thử chạy thành công.



Chu kỳ phát hành theo phương pháp lập trình cực đoan





Các thực hành lập trình cực đoan (a)

Nguyên tắc hoặc thực hành	Mô tả
Lập kế hoạch theo từng giai đoạn	Các yêu cầu được ghi lại trên thẻ câu chuyện và các câu chuyện sẽ được đưa vào bản phát hành được xác định dựa trên thời gian có sẵn và mức độ ưu tiên tương đối của chúng. Các nhà phát triển chia các câu chuyện này thành các "Nhiệm vụ" phát triển. Xem Hình 3.5 và 3.6.
Các bản phát hành nhỏ	Bộ chức năng hữu ích tối thiểu mang lại giá trị kinh doanh sẽ được phát triển trước tiên. Các bản phát hành của hệ thống diễn ra thường xuyên và bổ sung chức năng theo từng giai đoạn vào bản phát hành đầu tiên.
Thiết kế đơn giản	Thiết kế được thực hiện vừa đủ để đáp ứng các yêu cầu hiện tại và không hơn thế.
Phát triển theo phương pháp "kiểm thử trước"	Một khung kiểm thử đơn vị tự động được sử dụng để viết các bài kiểm thử cho một tính năng mới trước khi tính năng đó được triển khai.
Tái cấu trúc	Tất cả các nhà phát triển được kỳ vọng sẽ liên tục tái cấu trúc mã ngay khi phát hiện ra các cải tiến có thể thực hiện được. Điều này giúp mã nguồn luôn đơn giản và dễ bảo trì.

Các phương pháp lập trình cực đoan (b)



Lập trình theo cặp	Các nhà phát triển làm việc theo cặp, kiểm tra công việc của nhau và hỗ trợ lẫn nhau để luôn hoàn thành công việc một cách tốt nhất.
Sở hữu chung	Các cặp lập trình viên làm việc trên tất cả các lĩnh vực của hệ thống, để tránh hình thành các "hòn đảo chuyên môn" và đảm bảo tất cả các lập trình viên đều chịu trách nhiệm về toàn bộ mã nguồn. Bất kỳ ai cũng có thể thay đổi bất kỳ phần nào.
Tích hợp liên tục	Ngay khi công việc trên một tác vụ hoàn thành, nó sẽ được tích hợp vào toàn bộ hệ thống. Sau mỗi lần tích hợp như vậy, tất cả các bài kiểm tra đơn vị trong hệ thống phải đạt yêu cầu.
Tốc độ bền vững	Làm thêm giờ quá nhiều không được coi là chấp nhận được vì hiệu quả ròng thường là làm giảm chất lượng mã nguồn và năng suất trong trung hạn
Khách hàng tại chỗ	Một đại diện của người dùng cuối hệ thống (khách hàng) nên có mặt toàn thời gian để hỗ trợ nhóm XP. Trong quy trình lập trình cực đoan (XP), khách hàng là thành viên của nhóm phát triển và chịu trách nhiệm đưa các yêu cầu hệ thống đến nhóm để triển khai.

17

XP và các nguyên tắc linh hoạt



- Phát triển theo từng giai đoạn được hỗ trợ thông qua các bản phát hành hệ thống nhỏ và thường xuyên.
- Sự tham gia của khách hàng có nghĩa là sự gắn bó toàn thời gian của khách hàng với nhóm.
- Con người chứ không phải quy trình thông qua lập trình cặp, sở hữu chung và một quy trình tránh làm việc quá giờ.
- Thay đổi được hỗ trợ thông qua các đợt phát hành hệ thống định kỳ.
- Duy trì sự đơn giản thông qua việc tái cấu trúc mã liên tục.

18

Các phương pháp XP có ảnh hưởng



- Lập trình cực đoan (XP) tập trung vào khía cạnh kỹ thuật và không dễ dàng tích hợp với các thực hành quản lý trong phần lớn các tổ chức.
- Do đó, mặc dù phát triển linh hoạt sử dụng các thực hành từ XP, phương pháp này theo định nghĩa ban đầu không được sử dụng rộng rãi.
- Các thực hành chính
  - Câu chuyện người dùng để xác định yêu cầu
  - Tái cấu trúc
  - Phát triển theo phương pháp "Test-first"
  - Lập trình cặp

19

Câu chuyện người dùng cho các yêu cầu



- Trong XP, khách hàng hoặc người dùng là một phần của nhóm XP và chịu trách nhiệm đưa ra quyết định về các yêu cầu.
- Yêu cầu của người dùng được thể hiện dưới dạng câu chuyện người dùng hoặc kịch bản.
- Những nội dung này được ghi trên các thẻ và nhóm phát triển sẽ chia nhỏ chúng thành các tác vụ triển khai. Các tác vụ này là cơ sở để lập kế hoạch tiến độ và ước tính chi phí.
- Khách hàng chọn các câu chuyện để đưa vào bản phát hành tiếp theo dựa trên mức độ ưu tiên và ước tính tiến độ.

20

Một câu chuyện về "kê đơn thuốc"



Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

If you select 'current medication', you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.

21

Ví dụ về thẻ nhiệm vụ cho việc kê đơn thuốc



Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.
Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.
Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low.
If within the range, enable the 'Confirm' button.

22

Tái cấu trúc



- Quan điểm thông thường trong kỹ thuật phần mềm là thiết kế để thích ứng với sự thay đổi. Việc dành thời gian và công sức để dự đoán các thay đổi là đáng giá vì điều này giúp giảm chi phí ở các giai đoạn sau của vòng đời sản phẩm.
- Tuy nhiên, XP cho rằng điều này không đáng làm vì không thể dự đoán được những thay đổi một cách đáng tin cậy.
- Thay vào đó, XP đề xuất việc liên tục cải tiến mã nguồn (refactoring) để việc thực hiện các thay đổi trở nên dễ dàng hơn khi chúng thực sự cần thiết.

23

Tái cấu trúc



- Nhóm lập trình tìm kiếm các cơ hội cải tiến phần mềm và thực hiện những cải tiến này ngay cả khi chưa có nhu cầu cấp thiết.
- Điều này giúp phần mềm dễ hiểu hơn và do đó giảm bớt nhu cầu về tài liệu.
- Việc thực hiện các thay đổi trở nên dễ dàng hơn vì mã nguồn được cấu trúc tốt và rõ ràng.
- Tuy nhiên, một số thay đổi đòi hỏi phải tái cấu trúc kiến trúc và điều này tốn kém hơn nhiều.

24

Ví dụ về tái cấu trúc



- Tổ chức lại hệ thống phân cấp các lớp để loại bỏ mã trùng lặp.
- Sắp xếp lại và đổi tên các thuộc tính và phương thức để giúp chúng dễ hiểu hơn.
- Thay thế mã nhúng bằng các lệnh gọi đến các phương thức đã được đưa vào thư viện chương trình.

Phát triển theo phương pháp "Test-first"



- Kiểm thử là trọng tâm của XP và XP đã phát triển một phương pháp trong đó chương trình được kiểm thử sau mỗi lần thay đổi.
- Các đặc điểm kiểm thử của XP:
  - Phát triển theo phương pháp kiểm thử trước.
  - Phát triển kiểm thử theo từng giai đoạn dựa trên các kịch bản.
  - Sự tham gia của người dùng trong quá trình phát triển và xác thực kiểm thử.
  - Các bộ công cụ kiểm thử tự động được sử dụng để chạy tất cả các bài kiểm thử thành phần mỗi khi một phiên bản mới được xây dựng.

Phát triển hướng kiểm thử



- Viết các bài kiểm thử trước khi viết mã giúp làm rõ các yêu cầu cần triển khai.
- Các bài kiểm thử được viết dưới dạng chương trình thay vì dữ liệu để có thể được thực thi tự động. Bài kiểm thử bao gồm việc kiểm tra xem nó đã được thực thi chính xác hay chưa.
  - Thường dựa vào một khung thử nghiệm như Junit.
- Tất cả các bài kiểm thử trước đây và mới đều được chạy tự động khi có tính năng mới được thêm vào, từ đó đảm bảo rằng tính năng mới này không gây ra lỗi.

Sự tham gia của khách hàng



- Vai trò của khách hàng trong quá trình kiểm thử là hỗ trợ phát triển các bài kiểm thử chấp nhận cho các yêu cầu sẽ được triển khai trong phiên bản tiếp theo của hệ thống.
- Khách hàng là thành viên của nhóm sẽ viết các bài kiểm thử trong quá trình phát triển. Do đó, tất cả mã nguồn mới đều được kiểm tra để đảm bảo rằng nó đáp ứng đúng nhu cầu của khách hàng.
- Tuy nhiên, những người đảm nhận vai trò khách hàng thường có thời gian hạn chế và do đó không thể làm việc toàn thời gian cùng nhóm phát triển. Họ có thể cho rằng việc cung cấp yêu cầu đã là đóng góp đủ và do đó có thể miễn cưỡng tham gia vào quá trình kiểm thử.

Mô tả trường hợp kiểm thử để kiểm tra liều lượng



Test 4: Dose checking

Input:
1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:
1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:
OK or error message indicating that the dose is outside the safe range.

29

Tự động hóa kiểm thử



- Tự động hóa kiểm thử có nghĩa là các bài kiểm thử được viết dưới dạng các thành phần có thể thực thi trước khi nhiệm vụ được triển khai
  - Các thành phần kiểm thử này phải độc lập, phải mô phỏng việc nhập dữ liệu đầu vào cần kiểm thử và phải kiểm tra xem kết quả có đáp ứng các yêu cầu đầu ra hay không. Một khung công cụ kiểm thử tự động (ví dụ: JUnit) là hệ thống giúp việc viết các bài kiểm thử có thể thực thi và gửi một bộ bài kiểm thử để thực thi trở nên dễ dàng.
- Vì việc kiểm thử được tự động hóa, luôn có một tập hợp các bài kiểm thử có thể được thực thi nhanh chóng và dễ dàng
  - Bất cứ khi nào có chức năng nào được thêm vào hệ thống, các bài kiểm thử đều có thể được chạy và các vấn đề do mã mới gây ra có thể được phát hiện ngay lập tức.

30

Các vấn đề với phương pháp phát triển "test first"



- Các lập trình viên thường thích lập trình hơn là viết bài kiểm thử và đôi khi họ đi tắt khi viết bài kiểm thử. Ví dụ, họ có thể viết các bài kiểm thử chưa hoàn chỉnh, không kiểm tra hết tất cả các trường hợp ngoại lệ có thể xảy ra.
- Một số bài kiểm thử có thể rất khó để viết theo từng bước. Ví dụ, trong một giao diện người dùng phức tạp, thường rất khó để viết các bài kiểm thử đơn vị cho đoạn mã thực hiện "logic hiển thị" và luồng công việc giữa các màn hình.
- Rất khó để đánh giá mức độ đầy đủ của một bộ bài kiểm thử. Mặc dù bạn có thể có rất nhiều bài kiểm thử hệ thống, nhưng bộ bài kiểm thử của bạn có thể không đảm bảo độ bao phủ hoàn toàn.

31

Lập trình theo cặp



- Lập trình theo cặp là hình thức các lập trình viên làm việc theo cặp, cùng nhau phát triển mã nguồn.
- Điều này giúp xây dựng tinh thần đồng sở hữu mã nguồn và lan tỏa kiến thức trong toàn đội.
- Nó đóng vai trò như một quy trình kiểm tra không chính thức vì mỗi dòng mã đều được ít nhất 2 người xem xét.
- Phương pháp này khuyến khích việc tái cấu trúc (refactoring) vì toàn bộ đội ngũ đều được hưởng lợi từ việc cải thiện mã nguồn hệ thống.

32

Lập trình theo cặp



- Trong lập trình theo cặp, các lập trình viên ngồi cùng nhau trước một máy tính để phát triển phần mềm.
- Các cặp được hình thành một cách linh hoạt để đảm bảo tất cả các thành viên trong nhóm đều có cơ hội làm việc cùng nhau trong suốt quá trình phát triển.
- Việc chia sẻ kiến thức diễn ra trong quá trình lập trình theo cặp rất quan trọng vì nó giúp giảm thiểu rủi ro tổng thể cho dự án khi các thành viên trong nhóm rời đi.
- Lập trình theo cặp không nhất thiết là kém hiệu quả và có một số bằng chứng cho thấy rằng một cặp làm việc cùng nhau hiệu quả hơn so với hai lập trình viên làm việc riêng lẻ.



Quản lý dự án Agile

Quản lý dự án Agile



- Trách nhiệm chính của các nhà quản lý dự án phần mềm là quản lý dự án sao cho phần mềm được bàn giao đúng hạn và trong phạm vi ngân sách đã được lập kế hoạch cho dự án.
- Cách tiếp cận tiêu chuẩn trong quản lý dự án là theo hướng kế hoạch. Các nhà quản lý lập kế hoạch cho dự án, trong đó nêu rõ những gì cần được bàn giao, thời điểm bàn giao và ai sẽ tham gia phát triển các sản phẩm đầu ra của dự án.
- Quản lý dự án Agile đòi hỏi một phương pháp tiếp cận khác, phù hợp với phát triển theo từng giai đoạn và các thực hành được áp dụng trong các phương pháp Agile.

Scrum



- Scrum là một phương pháp Agile tập trung vào việc quản lý quá trình phát triển lặp lại thay vì các thực hành Agile cụ thể.
- Scrum bao gồm ba giai đoạn.
  - Giai đoạn đầu tiên là giai đoạn lập kế hoạch tổng quan, trong đó bạn xác định các mục tiêu chung của dự án và thiết kế kiến trúc phần mềm.
  - Tiếp theo là một chuỗi các chu kỳ sprint, trong đó mỗi chu kỳ phát triển một phần mở rộng của hệ thống.
  - Giai đoạn kết thúc dự án tổng kết dự án, hoàn thiện các tài liệu cần thiết như khung trợ giúp hệ thống và hướng dẫn sử dụng, đồng thời đánh giá những bài học kinh nghiệm rút ra từ dự án.

Thuật ngữ Scrum (a)



Thuật ngữ Scrum	Định nghĩa
Nhóm phát triển	Một nhóm các nhà phát triển phần mềm tự tổ chức, không nên có quá 7 người. Họ chịu trách nhiệm phát triển phần mềm và các tài liệu dự án thiết yếu khác.
Sản phẩm có thể giao được 'có khả năng phát hành'	Phần phần mềm được giao từ một sprint. Ý tưởng là phần phần mềm này phải nghĩa là nó đã ở trạng thái hoàn thiện và không cần thêm bất kỳ công việc nào, chẳng hạn như kiểm thử, để tích hợp vào . Trên thực tế, điều này không phải lúc nào cũng đạt được.
Danh sách công việc sản phẩm	Đây là danh sách các mục "cần làm" mà nhóm Scrum phải giải quyết. Chúng có thể là các định nghĩa tính năng cho phần mềm, yêu cầu phần mềm, câu chuyện người dùng hoặc mô tả các tác vụ bổ sung cần thiết, chẳng hạn như định nghĩa kiến trúc hoặc tài liệu hướng dẫn người dùng.
Chủ sở hữu sản phẩm	Một cá nhân (hoặc có thể là một nhóm nhỏ) có nhiệm vụ xác định các tính năng hoặc yêu cầu của sản phẩm, sắp xếp thứ tự ưu tiên cho việc phát triển và liên tục rà soát danh sách công việc tồn đọng của sản phẩm để đảm bảo rằng dự án tiếp tục đáp ứng các nhu cầu kinh doanh quan trọng. Chủ sở hữu sản phẩm có thể là khách hàng, nhưng cũng có thể là người quản lý sản phẩm trong một công ty phần mềm hoặc đại diện của các bên liên quan khác.

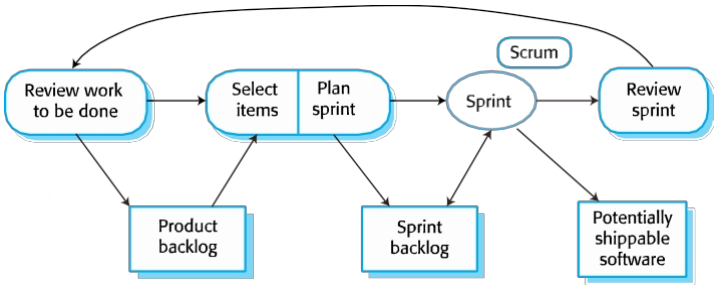
Thuật ngữ Scrum (b)



Thuật ngữ Scrum	Định nghĩa
Scrum	Cuộc họp hàng ngày của nhóm Scrum nhằm đánh giá tiến độ và sắp xếp thứ tự ưu tiên cho các công việc cần thực hiện trong ngày. Tốt nhất, đây nên là một cuộc họp trực tiếp ngắn gọn với sự tham gia của toàn bộ nhóm.
ScrumMaster	ScrumMaster chịu trách nhiệm đảm bảo quy trình Scrum được tuân thủ và hướng dẫn nhóm sử dụng Scrum một cách hiệu quả. Người này chịu trách nhiệm liên lạc với các bộ phận khác trong công ty và đảm bảo rằng nhóm Scrum không bị phân tâm bởi các yếu tố bên ngoài. Các nhà phát triển Scrum kiên quyết cho rằng không nên coi ScrumMaster là người quản lý dự án. Tuy nhiên, những người khác có thể không dễ dàng nhận ra sự khác biệt này.
Sprint	Một chu kỳ phát triển. Sprint thường kéo dài từ 2 đến 4 tuần.
Velocity	Một ước tính về lượng công việc trong danh sách công việc sản phẩm (product backlog) mà một nhóm có thể hoàn thành trong một sprint. Hiểu được tốc độ của nhóm sẽ giúp họ ước tính những gì có thể hoàn thành trong một sprint và cung cấp cơ sở để đo lường sự cải thiện hiệu suất.

Chu kỳ sprint Scrum





Chu kỳ Sprint trong Scrum



- Các sprint có thời gian cố định, thường là 2–4 tuần.
- Điểm khởi đầu cho việc lập kế hoạch là danh sách công việc tồn đọng của sản phẩm (product backlog), tức là danh sách các công việc cần thực hiện trong dự án.
- Giai đoạn lựa chọn có sự tham gia của toàn bộ đội ngũ dự án, họ làm việc cùng khách hàng để chọn ra các tính năng và chức năng từ danh sách công việc cần thực hiện (product backlog) để phát triển trong sprint.

Chu kỳ Sprint



- Sau khi các bên đã thống nhất, nhóm sẽ tự tổ chức để phát triển phần mềm.
- Trong giai đoạn này, nhóm làm việc tách biệt với khách hàng và tổ chức, với tất cả các kênh giao tiếp đều được chuyển qua người được gọi là 'Scrum Master'.
- Vai trò của Scrum Master là bảo vệ nhóm phát triển khỏi những yếu tố gây xao nhãng từ bên ngoài.
- Vào cuối sprint, công việc đã hoàn thành sẽ được rà soát và trình bày cho các bên liên quan. Sau đó, chu kỳ sprint tiếp theo sẽ bắt đầu.

41

Làm việc nhóm trong Scrum



- "Scrum master" là người điều phối, chịu trách nhiệm tổ chức các cuộc họp hàng ngày, theo dõi danh sách công việc cần thực hiện, ghi chép các quyết định, đánh giá tiến độ so với danh sách công việc và liên lạc với khách hàng cũng như ban quản lý bên ngoài nhóm.
- Toàn bộ đội ngũ tham gia các cuộc họp ngắn hàng ngày (Scrum), nơi tất cả thành viên chia sẻ thông tin, mô tả tiến độ kể từ cuộc họp trước, các vấn đề phát sinh và kế hoạch cho ngày tiếp theo.
  - Điều này có nghĩa là mọi thành viên trong nhóm đều nắm rõ tình hình và, nếu có vấn đề phát sinh, có thể điều chỉnh kế hoạch công việc ngắn hạn để ứng phó với chúng.

42

Lợi ích của Scrum

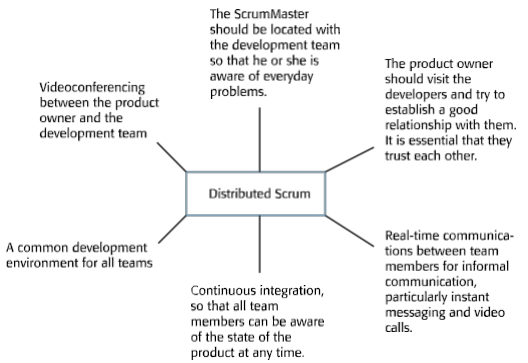


- Sản phẩm được chia nhỏ thành các phần dễ quản lý và dễ hiểu.
- Các yêu cầu không ổn định không cản trở tiến độ.
- Toàn bộ đội ngũ có cái nhìn tổng quan về mọi thứ và do đó, sự giao tiếp trong đội ngũ được cải thiện.
- Khách hàng nhận được các bản cập nhật đúng hạn và nhận được phản hồi về cách sản phẩm hoạt động.
- Sự tin tưởng giữa khách hàng và các nhà phát triển được thiết lập và một văn hóa tích cực được hình thành, trong đó mọi người đều kỳ vọng dự án sẽ thành công.

43

Scrum phân tán





44



Mở rộng các phương pháp Agile



Mở rộng quy mô các phương pháp Agile

- Các phương pháp Agile đã chứng minh được hiệu quả đối với các dự án quy mô nhỏ và vừa, có thể được phát triển bởi một nhóm nhỏ làm việc cùng địa điểm.
- Thỉnh thoảng có ý kiến cho rằng thành công của các phương pháp này xuất phát từ việc cải thiện giao tiếp, điều này chỉ có thể đạt được khi mọi người làm việc cùng nhau.
- Việc mở rộng quy mô các phương pháp Agile đòi hỏi phải điều chỉnh chúng để phù hợp với các dự án lớn hơn, kéo dài hơn, nơi có nhiều nhóm phát triển, có thể làm việc tại các địa điểm khác nhau.



Mở rộng theo chiều ngang và mở rộng theo dọc

- 'Mở rộng theo chiều dọc' liên quan đến việc áp dụng các phương pháp linh hoạt để phát triển các hệ thống phần mềm quy mô lớn mà một nhóm nhỏ không thể thực hiện được.
- 'Mở rộng theo chiều ngang' tập trung vào việc áp dụng các phương pháp linh hoạt trong một tổ chức lớn có nhiều năm kinh nghiệm phát triển phần mềm.
- Khi mở rộng quy mô các phương pháp linh hoạt, điều quan trọng là phải duy trì các nguyên tắc cơ bản của phương pháp linh hoạt:
  - Lập kế hoạch linh hoạt, phát hành hệ thống thường xuyên, tích hợp liên tục, phát triển hướng kiểm thử và giao tiếp tốt trong đội ngũ.



Các vấn đề thực tiễn với các phương pháp A

- Tính không chính thức của phát triển Agile không tương thích với cách tiếp cận pháp lý trong việc định nghĩa hợp đồng thường được sử dụng tại các công ty lớn.
- Các phương pháp Agile phù hợp nhất cho việc phát triển phần mềm mới hơn là bảo trì phần mềm. Tuy nhiên, phần lớn chi phí phần mềm trong các công ty lớn lại đến từ việc bảo trì các hệ thống phần mềm hiện có.
- Các phương pháp Agile được thiết kế cho các nhóm nhỏ làm việc cùng địa điểm, nhưng hiện nay phần lớn các dự án phát triển phần mềm đều liên quan đến các nhóm phân tán trên toàn thế giới.

Các vấn đề liên quan đến hợp đồng



- Hầu hết các hợp đồng phần mềm cho các hệ thống tùy chỉnh đều dựa trên một bản mô tả yêu cầu, trong đó nêu rõ những gì nhà phát triển hệ thống phải triển khai cho khách hàng.
- Tuy nhiên, điều này ngăn cản việc lồng ghép giữa việc lập bản mô tả yêu cầu và quá trình phát triển — vốn là tiêu chuẩn trong phương pháp phát triển linh hoạt (agile).
- Cần có một hợp đồng thanh toán dựa trên thời gian làm việc của nhà phát triển thay vì dựa trên chức năng.
  - Tuy nhiên, điều này được nhiều bộ phận pháp lý coi là rủi ro cao vì không thể đảm bảo được những gì phải được bàn giao.

Phương pháp Agile và bảo trì phần mềm



- Hầu hết các tổ chức chi tiêu nhiều hơn cho việc bảo trì phần mềm hiện có so với phát triển phần mềm mới. Do đó, nếu các phương pháp Agile muốn thành công, chúng phải hỗ trợ cả việc bảo trì lẫn phát triển ban đầu.
- Hai vấn đề chính:
  - Các hệ thống được phát triển theo phương pháp Agile có thể bảo trì được không, khi quá trình phát triển tập trung vào việc giảm thiểu tài liệu chính thức?
  - Các phương pháp Agile có thể được sử dụng hiệu quả để phát triển hệ thống nhằm đáp ứng các yêu cầu thay đổi của khách hàng không?
- Các vấn đề có thể phát sinh nếu đội ngũ phát triển ban đầu không thể được duy trì.

Bảo trì theo phương pháp Agile



- Các vấn đề chính là:
  - Thiếu tài liệu sản phẩm
  - Giữ khách hàng tham gia vào quá trình phát triển
  - Duy trì tính liên tục của đội ngũ phát triển
- Phát triển linh hoạt (Agile) phụ thuộc vào việc đội ngũ phát triển phải nắm rõ và hiểu rõ những gì cần phải làm.
- Đối với các hệ thống có vòng đời dài, đây là một vấn đề thực sự vì các nhà phát triển ban đầu sẽ không phải lúc nào cũng làm việc trên hệ thống đó.

Phương pháp Agile và phương pháp dựa trên kế hoạch



- Hầu hết các dự án đều bao gồm các yếu tố của quy trình theo kế hoạch và quy trình Agile. Việc quyết định sự cân bằng phụ thuộc vào:
  - Liệu việc có một bản mô tả yêu cầu và thiết kế chi tiết trước khi chuyển sang giai đoạn triển khai có quan trọng không? Nếu có, bạn có thể cần sử dụng phương pháp dựa trên kế hoạch.
  - Liệu chiến lược giao hàng theo từng giai đoạn, trong đó bạn giao phần mềm cho khách hàng và nhận phản hồi nhanh chóng từ họ, có thực tế không? Nếu có, hãy xem xét sử dụng các phương pháp Agile.
  - Hệ thống đang được phát triển có quy mô lớn đến mức nào? Các phương pháp linh hoạt hiệu quả nhất khi hệ thống có thể được phát triển bởi một nhóm nhỏ làm việc cùng địa điểm và có thể giao tiếp một cách không chính thức. Điều này có thể không khả thi đối với các hệ thống lớn đòi hỏi đội ngũ phát triển lớn hơn, do đó có thể phải sử dụng phương pháp dựa trên kế hoạch.

Các nguyên tắc Agile và thực tiễn tổ chức



Nguyên tắc	Thực tiễn
Sự tham gia của khách hàng	Điều này phụ thuộc vào việc có một khách hàng sẵn sàng và có khả năng dành thời gian làm việc cùng đội ngũ phát triển, đồng thời có thể đại diện cho tất cả các bên liên quan của hệ thống. Thường thì các đại diện của khách hàng có những yêu cầu khác về thời gian và không thể tham gia đầy đủ vào quá trình phát triển phần mềm. Khi có các bên liên quan bên ngoài, chẳng hạn như cơ quan quản lý, sẽ rất khó để truyền đạt quan điểm của họ đến nhóm Agile.
Chấp nhận sự thay đổi	Việc sắp xếp thứ tự ưu tiên cho các thay đổi có thể cực kỳ khó khăn, đặc biệt là trong các hệ thống có nhiều bên liên quan. Thông thường, mỗi bên liên quan sẽ đặt mức độ ưu tiên khác nhau cho các thay đổi khác nhau.
Phân phối theo từng giai đoạn	Các chu kỳ lặp lại nhanh và kế hoạch phát triển ngắn hạn không phải lúc nào cũng phù hợp với các chu kỳ lập kế hoạch dài hạn của kế hoạch kinh doanh và tiếp thị. Các nhà quản lý tiếp thị có thể cần biết các tính năng sản phẩm trước vài tháng để chuẩn bị một chiến dịch tiếp thị hiệu quả.

53

Các nguyên tắc Agile và thực tiễn tổ chức

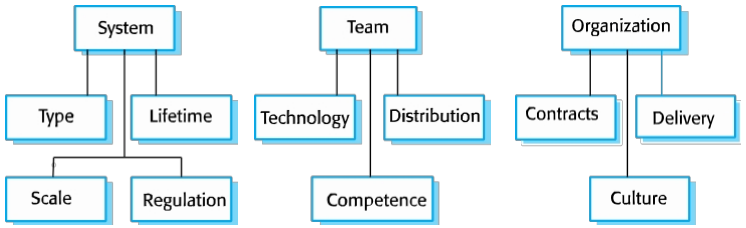


Nguyên tắc	Thực tiễn
Duy trì sự đơn giản	Dưới áp lực của tiến độ triển khai, các thành viên trong nhóm có thể không có thời gian để thực hiện các biện pháp đơn giản hóa hệ thống cần thiết.
Con người chứ không phải quy trình	Các thành viên trong nhóm có thể không có tính cách phù hợp với mức độ tham gia sâu sắc vốn là đặc trưng của các phương pháp Agile, và do đó có thể không tương tác tốt với các thành viên khác trong nhóm.

54

Các yếu tố linh hoạt và dựa trên kế hoạch





55

Các vấn đề về hệ thống



- Hệ thống đang được phát triển có quy mô lớn đến mức nào?
  - Các phương pháp Agile hiệu quả nhất khi áp dụng cho một nhóm nhỏ làm việc cùng địa điểm và có thể giao tiếp một cách thân mật.
- Loại hệ thống đang được phát triển là gì?
  - Các hệ thống đòi hỏi nhiều phân tích trước khi triển khai cần có thiết kế khá chi tiết để thực hiện phân tích này.
- Tuổi thọ dự kiến của hệ thống là bao lâu?
  - Các hệ thống có tuổi thọ dài cần có tài liệu để truyền đạt ý định của các nhà phát triển hệ thống đến đội ngũ hỗ trợ.
- Hệ thống có chịu sự điều chỉnh của các quy định bên ngoài không?
  - Nếu hệ thống bị quy định, bạn có thể sẽ phải lập tài liệu chi tiết như một phần của hồ sơ an toàn hệ thống.

56

Con người và đội ngũ



- Các nhà thiết kế và lập trình viên trong nhóm phát triển có trình độ như thế nào?
  - Thỉnh thoảng có ý kiến cho rằng các phương pháp Agile đòi hỏi trình độ kỹ năng cao hơn so với các phương pháp dựa trên kế hoạch, trong đó các lập trình viên chỉ đơn thuần chuyển đổi thiết kế chi tiết thành mã nguồn.
- Đội ngũ phát triển được tổ chức như thế nào?
  - Có thể cần các tài liệu thiết kế nếu đội ngũ làm việc từ xa.
- Có những công nghệ hỗ trợ nào sẵn có?
  - Hỗ trợ từ IDE cho việc trực quan hóa và phân tích chương trình là điều cần thiết nếu không có tài liệu thiết kế.

Các vấn đề về tổ chức



- Các tổ chức kỹ thuật truyền thống có văn hóa phát triển dựa trên kế hoạch, vì đây là tiêu chuẩn trong lĩnh vực kỹ thuật.
- Việc lập một bản đặc tả hệ thống chi tiết có phải là thực hành tiêu chuẩn của tổ chức không?
- Liệu đại diện khách hàng có sẵn sàng cung cấp phản hồi về các giai đoạn phát triển hệ thống hay không?
- Phương pháp phát triển linh hoạt không chính thức có thể phù hợp với văn hóa tổ chức chú trọng tài liệu chi tiết không?

Các phương pháp Agile cho các hệ thống lớ



- Các hệ thống quy mô lớn thường là tập hợp của các hệ thống riêng biệt, có khả năng giao tiếp với nhau, trong đó mỗi hệ thống được phát triển bởi các nhóm riêng biệt. Thông thường, các nhóm này làm việc tại các địa điểm khác nhau, đôi khi ở các múi giờ khác nhau.
- Các hệ thống quy mô lớn là "hệ thống brownfield", tức là chúng bao gồm và tương tác với một số hệ thống hiện có. Nhiều yêu cầu hệ thống liên quan đến sự tương tác này và do đó không thực sự phù hợp với tính linh hoạt và phát triển theo từng giai đoạn.
- Khi nhiều hệ thống được tích hợp để tạo thành một hệ thống, một phần đáng kể của quá trình phát triển tập trung vào việc cấu hình hệ thống hơn là phát triển mã nguồn mới.

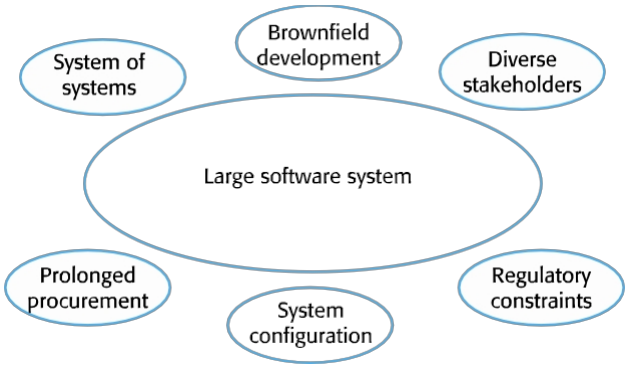
Phát triển hệ thống quy mô lớn



- Các hệ thống quy mô lớn và quy trình phát triển của chúng thường bị ràng buộc bởi các quy định và quy tắc bên ngoài, hạn chế cách thức phát triển của chúng.
- Các hệ thống quy mô lớn có thời gian mua sắm và phát triển kéo dài. Việc duy trì các đội ngũ thống nhất và am hiểu về hệ thống trong suốt khoảng thời gian đó là rất khó khăn, bởi vì không thể tránh khỏi việc nhân sự chuyển sang các công việc và dự án khác.
- Các hệ thống quy mô lớn thường có một nhóm các bên liên quan đa dạng. Trên thực tế, việc thu hút tất cả các bên liên quan khác nhau này tham gia vào quy trình phát triển là điều gần như không thể.

Các yếu tố trong các hệ thống quy mô lớn

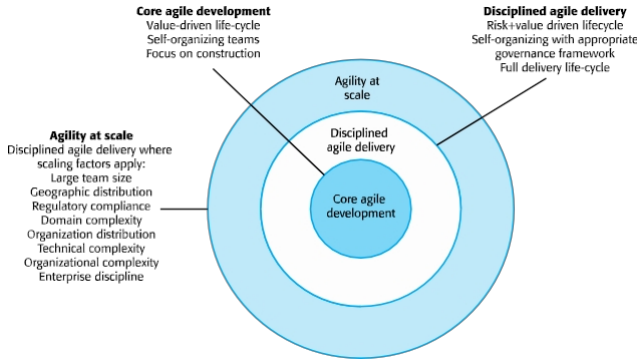




61

Mô hình linh hoạt quy mô lớn của IBM





62

Mở rộng quy mô lên các hệ thống lớn



- Một phương pháp hoàn toàn theo từng bước trong công tác phân tích yêu cầu là không thể.
- Không thể chỉ có một chủ sở hữu sản phẩm hoặc đại diện khách hàng duy nhất.
- Trong phát triển hệ thống quy mô lớn, không thể chỉ tập trung vào mã nguồn của hệ thống.
- Cần thiết kế và áp dụng các cơ chế giao tiếp liên nhóm.
- Tích hợp liên tục về mặt thực tế là không thể. Tuy nhiên, việc duy trì các bản dựng hệ thống thường xuyên và các đợt phát hành định kỳ của hệ thống là điều thiết yếu.

63

Scrum đa nhóm



- *Sao chép vai trò*
  - Mỗi nhóm có một Product Owner phụ trách phần công việc của mình và một ScrumMaster.
- *Kiến trúc sư sản phẩm*
  - Mỗi nhóm chọn một kiến trúc sư sản phẩm và các kiến trúc sư này hợp tác để thiết kế và phát triển kiến trúc hệ thống tổng thể.
- *Đồng bộ hóa các đợt phát hành*
  - Các mốc thời gian phát hành sản phẩm của từng nhóm được đồng bộ hóa để đảm bảo tạo ra một hệ thống hoàn chỉnh và có thể trình diễn được.
- *Scrum of Scrums*
  - Hàng ngày có một cuộc họp Scrum of Scrums, nơi các đại diện từ mỗi nhóm gặp gỡ để thảo luận về tiến độ và lập kế hoạch cho công việc cần thực hiện.

64

Các phương pháp Agile trong các tổ chức



- Các nhà quản lý dự án chưa có kinh nghiệm về các phương pháp Agile có thể do dự khi chấp nhận rủi ro của một phương pháp tiếp cận mới.
- Các tổ chức lớn thường có các quy trình và tiêu chuẩn chất lượng mà tất cả các dự án đều phải tuân thủ, và do bản chất quan liêu của chúng, những quy trình và tiêu chuẩn này có khả năng không tương thích với các phương pháp Agile.
- Các phương pháp Agile dường như phát huy hiệu quả tốt nhất khi các thành viên trong nhóm có trình độ kỹ năng tương đối cao. Tuy nhiên, trong các tổ chức lớn, trình độ kỹ năng và năng lực của các thành viên thường rất đa dạng.
- Có thể sẽ có sự phản kháng về mặt văn hóa đối với các phương pháp linh hoạt, đặc biệt là tại những tổ chức có truyền thống lâu đời trong việc áp dụng các quy trình kỹ thuật hệ thống truyền thống.

Điểm chính



- Các phương pháp Agile là các phương pháp phát triển theo từng giai đoạn, tập trung vào việc phát triển phần mềm nhanh chóng, phát hành phần mềm thường xuyên, giảm thiểu gánh nặng quy trình bằng cách tối thiểu hóa tài liệu và tạo ra mã nguồn chất lượng cao.
- Các thực hành phát triển Agile bao gồm
  - Câu chuyện người dùng để xác định yêu cầu hệ thống
  - Phát hành phần mềm thường xuyên,
  - Cải tiến phần mềm liên tục
  - Phát triển theo phương pháp "kiểm thử trước"
  - Sự tham gia của khách hàng trong nhóm phát triển.

Điểm chính



- Scrum là một phương pháp linh hoạt cung cấp một khung quản lý dự án.
  - Phương pháp này xoay quanh một chuỗi các đợt sprint, là những khoảng thời gian cố định để phát triển các phần mở rộng của hệ thống.
- Nhiều phương pháp phát triển thực tiễn là sự kết hợp giữa phát triển dựa trên kế hoạch và phát triển linh hoạt.
- Việc mở rộng các phương pháp linh hoạt cho các hệ thống lớn là một thách thức.
  - Các hệ thống quy mô lớn cần thiết kế từ đầu và một số tài liệu cũng như thực tiễn tổ chức có thể mâu thuẫn với tính linh hoạt của các phương pháp Agile.